

Praxis: Speicherverwaltung beobachten

Vom Systemzustand bis zur einzelnen Virtual Memory Area (VMA)

3.06

Das Lab folgt drei Ebenen der Beobachtung

01 Systemweit

Wie viel Speicher ist verfügbar
und wofür wird er verwendet?

02 Prozessbezogen

Welche virtuellen Bereiche besitzt
ein ausgewählter Prozess?

03 Kernel Intern

Welche großen vmalloc-Bereiche
sind im Kernel sichtbar?

Ergebnis

Beobachtungen mit Messwert, Befehl und Interpretation dokumentieren.

Messwerte brauchen einen definierten Ausgangspunkt

01 Kernel

```
uname -r
```

02 Zeitpunkt

```
date --iso-8601=seconds
```

03 System

```
cat /proc/meminfo
```

04 Prozess

```
PID=$$
```

05 Berechtigung

```
lesbarer /proc-Zugriff
```

Nur lesen: In diesem Lab werden keine /proc/sys/vm-Parameter verändert.

Vier Dateien beantworten vier verschiedene Fragen

```
/proc/meminfo
```

Systemweite Speicherstatistik

```
/proc/<pid>/maps
```

Virtuelle Bereiche und Rechte

```
/proc/<pid>/smaps
```

Verbrauch je Mapping

```
/proc/vmallocinfo
```

Kernel-vmalloc-Bereiche

Nicht vermischen: Gesamtzustand, Adressraum und Resident Set sind unterschiedliche Perspektiven.

/proc/meminfo zeigt den globalen Speicherzustand

```
$ grep -E 'MemTotal|MemFree|MemAvailable|Buffers|Cached|Swap' /proc/meminfo
```

Kapazität

MemTotal

Nutzbarer physischer Hauptspeicher.

Sofort frei

MemFree

Derzeit vollständig unbenutzte Pages.

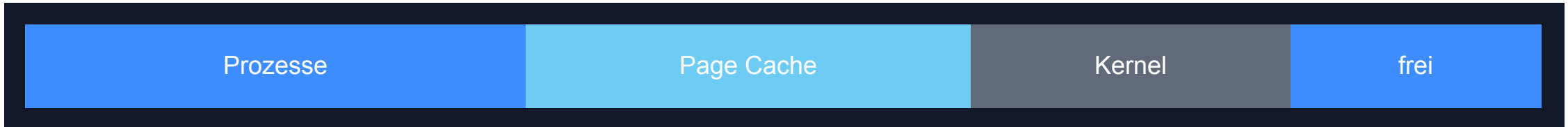
Schätzung

MemAvailable

Speicher, der voraussichtlich ohne Swap verfügbar ist.

Wichtig „Wenig frei“ bedeutet nicht automatisch „Speichermangel“ – Cache ist grundsätzlich rückgewinnbar.

MemFree allein ist eine schwache Kennzahl



MemFree

nur der aktuell unbenutzte Teil

MemAvailable

berücksichtigt einen Teil des rückgewinnbaren Speichers

Interpretation: Verfügbarkeit und Rückgewinnbarkeit gemeinsam betrachten.

Differenzen machen Momentaufnahmen aussagekräftig

Baseline

```
$ cp /proc/meminfo /tmp/meminfo.before
```

Last erzeugen

```
$ python3 -c 'a=bytearray(64*1024*1024);  
input()'
```

Danach

```
$ diff -u /tmp/meminfo.before /proc/meminfo
```

Beobachten

- Welche Felder ändern sich?
- Welche bleiben nahezu stabil?
- Welche Änderung war unerwartet?

Messungen schwanken: Prozessaktivität und Cache-Verhalten laufen parallel weiter.

Ein PID verbindet Prozess und Speicher

```
$ PID=$$
```

Aktuelle Shell

```
$ ps -p $PID -o pid,comm,rss,vsz
```

Schneller Überblick

```
$ readlink /proc/$PID/exe
```

Ausführbare Datei

```
$ cat /proc/$PID/status
```

Status und ausgewählte Speicherwerte

maps beschreibt Lage, Rechte und Herkunft jeder VMA

```
$ cat /proc/$PID/maps  
  
7f2a9c000000-7f2a9c021000 rw-p 00000000 00:00 0 [heap]
```

- | | | | | | |
|----|-------------------------------|----|--|----|---------------------------------|
| 01 | Adressbereich
Start–Ende | 02 | Rechte
r w x · privat/geteilt | 03 | Offset
Position in der Datei |
| 04 | Gerät + Inode
Backing File | 05 | Pfad / Name
Datei oder Spezialbereich | | |

Mappingtypen zeigen die Herkunft des Speichers

[stack]

Thread-/Prozessstack

rw-p

[heap]

klassischer Prozessheap

rw-p

Shared Library

dateibasiert

r-xp / r--p / rw-p

anonym

mmap ohne Datei

rw-p

shared

zwischen Prozessen teilbar

rw-s

Aufgabe: Je einen anonymen, dateibasierten und speziellen Bereich markieren.

smaps ergänzt jede VMA um ihren tatsächlichen Verbrauch

```
$ awk '/^Size:|^Rss:|^Pss:|^Private_|^Shared_/{print}' /proc/$PID/smaps | head -30
```

Virtuell

Size

Größe des gesamten Mappings.

Resident

Rss

Virtuelle Speicher-Pages die aktuell im physischen RAM verfügbar sind.

Gewichtet

Pss

Über Prozesse geteilte Speicher-Pages anteilig zugerechnet.

Eigentum

Private_*

Nur diesem Prozess zugerechnete Pages.

Size ≥ Rss

Eine große VMA muss nicht vollständig resident sein.

RSS und PSS zählen geteilte Seiten anders

RSS

Jeder Prozess zählt die gesamte residente gemeinsame Seite.

Gut für: residenter Umfang eines einzelnen Prozesses

PSS

Gemeinsame Seiten werden durch die Anzahl ihrer Mapper geteilt.

Gut für: fairere Zuordnung in Prozessgruppen

Beispiel: 12 MiB geteilt durch 3 Prozesse → jeder erhält 4 MiB PSS.

vmallocinfo öffnet den Blick auf den Kerneladressraum

```
$ sudo head -20 /proc/vmallocinfo
```

Berechtigungen können eingeschränkt sein.

01 Adressbereich

virtueller Start und Ende

02 Größe

reservierter virtueller
Umfang

03 Aufrufer

Ort der Allokation, sofern
sichtbar

```
$ sudo sort -k2 -n /proc/vmallocinfo | tail
```

Leitfrage: Welche Einträge belegen die größten virtuellen Bereiche?

Die Beobachtung endet mit einer begründeten Aussage

01 Beobachtung

Messwert oder konkrete Mappingzeile notieren.

02 Einordnung

System, Prozess oder einzelne VMA benennen.

03 Erklärung

Virtuell, resident, privat oder geteilt unterscheiden.

04 Grenze

Schwankung, Berechtigung oder Messaufwand angeben.

Nächster Schritt: Speicher im Kernel mit der passenden API allokalieren.